

Trabajo Práctico Obligatorio (TPO)

1. Objetivo General

Diseñar e implementar una solución en Java para una problemática de complejidad media. El sistema debe integrar múltiples Tipos Abstractos de Datos (TDAs), seleccionados y justificados según su rendimiento y adecuación al problema.

2. Metodología y Seguimiento (Git)

- Grupos: 2 a 3 integrantes.
- Repositorio: El uso de un repositorio (GitHub/GitLab) es obligatorio.
- Seguimiento Individual: La nota final dependerá del análisis de los *commits*. Cada estudiante debe reflejar su trabajo de manera modular en el historial de versiones; contribuciones masivas o de un solo autor al final del proyecto no serán válidas para acreditar la participación individual.

3. Cronograma de Entregas

- Etapa 1 (Propuesta y Diseño - 25 de Mayo): Presentación del problema elegido, lógica de resolución y diagramas de las estructuras a utilizar.
- Etapa 2 (Avance Funcional – 08 de Junio): Demostración de los módulos principales en el repositorio Git, incluyendo la implementación de las estructuras base.
- Etapa 3 (Presentación Final – 18 de Junio): Entrega del código completo y casos de prueba.

4. Alternativas de Problemática (Elegir una)

Alternativa A: Ecosistema de Red Social Profesional

Descripción: Crear una plataforma que gestione perfiles, conexiones y el flujo de información laboral.

- Identificación inmediata: Se debe recuperar el perfil de cualquier usuario de forma instantánea mediante un identificador único, evitando búsquedas secuenciales.
- Red de Contactos: El sistema debe representar las conexiones entre profesionales. Es vital poder calcular el grado de separación entre dos personas y sugerir "contactos recomendados" basados en vínculos comunes.
- Jerarquía de Habilidades: Las competencias laborales deben organizarse jerárquicamente (ej: Tecnología > Desarrollo > Java) para realizar búsquedas por especialidad.
- Gestión de Postulaciones: Las solicitudes de empleo deben procesarse siguiendo estrictamente el orden de llegada.
- Historial de Cambios: El sistema debe permitir deshacer la última actualización del perfil si el usuario lo solicita.

Alternativa B: Sistema Inteligente de Tráfico y Emergencias

Descripción: Administrar la infraestructura vial y la respuesta ante incidentes en una ciudad inteligente.

- **Modelado de la Ciudad:** Se debe representar la red de calles e intersecciones, permitiendo determinar si existe una ruta entre dos puntos y cuál es la más corta en términos de distancia o tiempo.
- **Despacho de Emergencias:** Ante múltiples accidentes, el sistema debe asignar asistencia priorizando la gravedad del evento sobre el orden cronológico del reporte.
- **Implementar un mecanismo de indexación** para el parque de dispositivos (semáforos y cámaras). El sistema debe ser capaz de recuperar o modificar el estado de una unidad mediante su código único en tiempo constante, garantizando que el tiempo de respuesta no aumente linealmente con el volumen de datos registrados
- **Organización Territorial:** La ciudad se divide en zonas, barrios y manzanas; el sistema debe reflejar esta estructura de contenedores anidados para reportes estadísticos.
- **Flujo Vehicular:** Los vehículos en una intersección deben ser liberados según el orden en que llegaron a la misma.

Alternativa C: Centro Logístico de Distribución Avanzada

Descripción: Gestionar el almacenamiento, stock y rutas de despacho de una cadena de suministros.

- **Localización de Stock:** Cada producto posee un código universal; el sistema debe hallar su ubicación en el depósito sin realizar recorridos lineales por las estanterías.
- **Optimización de Recolección:** Los operarios deben recoger productos en un orden que minimice el recorrido dentro del almacén, basándose en la interconexión de los pasillos.
- **Control de Inventario Crítico:** Los productos deben mantenerse organizados de tal forma que se puedan identificar rápidamente aquellos con menor existencia para su reposición inmediata.
- **Línea de Expedición:** Los pedidos listos deben esperar en una fila para ser cargados en los camiones de reparto.
- **Trazabilidad de Lotes:** Debe existir un registro de los últimos movimientos de mercadería que permita revertir una acción errónea en el inventario.

5. Requerimientos Técnicos y de Calidad

- **Eficiencia:** El código debe ser optimizado. Se valorará el uso de estructuras que reduzcan la complejidad temporal de las operaciones críticas
- **Robustez:** La aplicación debe manejar errores de entrada y casos límite (ej: buscar rutas inexistentes o IDs duplicados).
- **Casos de Prueba:** Se debe presentar un documento con las pruebas realizadas que garanticen el correcto funcionamiento de cada funcionalidad solicitada.
- **Legibilidad:** El código debe estar modularizado, con nombres de variables descriptivos y comentarios que expliquen la lógica compleja.

